

CALCULADORA CIENTÍFICA

Documentação do Projeto

Desenvolvimento de Software — Python 3 | Tkinter

Versão 1.0 — 2026

Esta documentação segue o Mapa de Documentação/Projeto definido pelo professor, incluindo as seções obrigatórias nas páginas e formatos corretos.

INTEGRANTES DO GRUPO

#	Nome Completo	RGM
1	Alexander Rosa Ribeiro	_____
2	Carlos Gomes de Souza Junior	_____
3	Caua Francisco Gomes da Silva	_____
4	Guilherme Pinheiro Paulino	_____
5	Valdeir Almeida Silva	_____
6	Samuel Freitas Brito	_____
7	Vinicius da Silva Rocha	_____

** Nomes em ordem alfabética. RGM em letras maiúsculas.*

Resumo

Este projeto consiste no desenvolvimento de uma Calculadora Científica completa em Python com a biblioteca gráfica Tkinter. O programa oferece interface visual intuitiva com operações aritméticas básicas e cálculos trigonométricos, logarítmicos e exponenciais avançados, modo DEG/RAD, função 2nd para inversas trigonométricas, suporte a teclado físico e ajuste dinâmico de fonte no display.

Objetivos do Programa

- Implementar interface gráfica completa com display duplo (expressão e resultado);
- Desenvolver as quatro operações aritméticas básicas;
- Incorporar funções científicas: sin, cos, tan (e inversas), log, ln, exp, 10^x , x^2 , n!;
- Suporte aos modos DEG (graus) e RAD (radianos) com alternância por botão;
- Modo secundário 2nd para acesso rápido às funções trigonométricas inversas;
- Tratamento de erros: divisão por zero, operações inválidas, limites de entrada;
- Entrada pelo teclado físico (números, operadores, Enter, Backspace, Escape);
- Ajuste dinâmico de tamanho de fonte conforme comprimento do resultado.

Arquitetura do Programa

O programa foi estruturado em três camadas independentes que se comunicam por variáveis de estado e chamadas de função:

Camada	Componente	Responsabilidade
1 — Visual	Frame display + grid 7x5 de botões	Tkinter / Interface
2 — Estado	expressao, modo_graus, segundo_modos	Variáveis globais
3 — Lógica	adicionar(), calcular(), aplicar_func(), tecla()	Funções Python

Decisões de Design

eval(): Avalia expressões compostas como $2+3*4$ sem precisar de um parser próprio.

Limite 32 chars: Evita travamentos e mantém o display sempre legível.

ajustar_fonte(): Reduz automaticamente o tamanho da fonte conforme o texto cresce no display.

Modo 2nd: Altera sin/cos/tan para asin/acos/atan sem precisar de botões extras.

janela.bind(): Captura eventos do teclado físico, incluindo o teclado numérico (KP_*).

Paleta de cores: Tons escuros com laranja nos operadores para melhor usabilidade e contraste.

Fluxo de Funcionamento

Quando o usuário clica em um botão numérico ou pressiona uma tecla, a função `adicionar()` concatena o símbolo à variável `'expressao'` e atualiza o `display`. Ao pressionar `'='`, a função `calcular()` passa a string para `eval()`, que retorna o resultado matemático. Funções científicas como `sin()` e `log()` recebem o valor atual da expressão, aplicam a operação matemática via biblioteca `math` e exibem o resultado diretamente no `display`.

Tratamento de Erros

- `ZeroDivisionError`: capturado separadamente, exibe `'Div/0!'` ao usuário;
- Exceção genérica: captura qualquer outra expressão inválida, exibe `'Erro'`;
- Limite de caracteres: a função `adicionar()` bloqueia entrada após 32 caracteres;
- Funções matematicamente inválidas (ex: `sqrt` de negativo): tratadas pelo `except` geral.

Botões e Layout

A grade de botões usa 7 linhas por 5 colunas, gerenciada pelo sistema `grid` do Tkinter. A função auxiliar `btn()` encapsula a criação de cada `Button`, recebendo posição, cor e comando como parâmetros, eliminando repetição de código. Os botões científicos ficam na coluna 0 (azul escuro), os numéricos nas colunas 1 a 3 (cinza) e os operadores na coluna 4 (laranja).

CÓDIGO FONTE — cal.py

Arquivo: cal.py | Python 3 | Tkinter + math | Versão 1.0 — 2026

```
from tkinter import *
import math

# Cores
cor1 = "#3b3b3b" # Preto
cor2 = "#ffffff" # Branco
cor3 = "#38576b" # Azul (display)
cor4 = "#7d7c7c" # Cinza (botoes normais)
cor5 = "#FFAB40" # Laranja (operadores / =)
cor6 = "#2a6496" # Azul escuro (botoes cientificos)
cor7 = "#5a5a5a" # Cinza medio

janela = Tk()
janela.title("Calculadora Cientifica")
janela.geometry("320x500")
janela.config(bg=cor1)
janela.resizable(False, False)

# Variaveis de estado
expressao = ""
modo_graus = True
segundo_mod0 = False

# Display
frame_tela = Frame(janela, width=320, height=80, bg=cor3)
frame_tela.grid(row=0, column=0, sticky="ew")
frame_tela.grid_propagate(False)

display_expr = StringVar(value="")
display_res = StringVar(value="0")

lbl_expr = Label(frame_tela, textvariable=display_expr,
font=("Courier", 11), bg=cor3, fg="#aac8d8", anchor="e")
lbl_expr.place(x=0, y=6, width=314, height=18)

lbl_res = Label(frame_tela, textvariable=display_res,
font=("Courier", 28, "bold"), bg=cor3, fg=cor2, anchor="e")
lbl_res.place(x=0, y=28, width=314, height=46)

def ajustar_fonte(texto):
    n = len(str(texto))
    if n <= 9: tam = 28
    elif n <= 13: tam = 22
    elif n <= 18: tam = 16
    elif n <= 24: tam = 12
    else: tam = 9
    lbl_res.config(font=("Courier", tam, "bold"))

MAX_CHARS = 32

def adicionar(simbolo):
```

```

global expressao
if len(expressao) >= MAX_CHARS:
    return
expressao += str(simbolo)
display_expr.set(expressao)
display_res.set(expressao)
ajustar_fonte(expressao)

def limpar():
    global expressao
    expressao = ""
    display_expr.set("")
    display_res.set("0")
    ajustar_fonte("0")

def apagar():
    global expressao
    expressao = expressao[:-1]
    display_expr.set(expressao)
    val = expressao if expressao else "0"
    display_res.set(val)
    ajustar_fonte(val)

def calcular():
    global expressao
    try:
        resultado = eval(expressao)
        if isinstance(resultado, float) and resultado.is_integer():
            resultado = int(resultado)
        s = str(resultado)
        display_expr.set(expressao + " =")
        display_res.set(s)
        ajustar_fonte(s)
        expressao = s
    except ZeroDivisionError:
        display_res.set("Div/0!")
        ajustar_fonte("Div/0!")
        expressao = ""
    except Exception:
        display_res.set("Erro")
        ajustar_fonte("Erro")
        expressao = ""

def _angulo(x):
    return math.radians(x) if modo_graus else x

def _de_rad(x):
    return math.degrees(x) if modo_graus else x

def aplicar_func(func_nome):
    global expressao
    try:
        val = float(eval(expressao)) if expressao else 0.0
        if func_nome == "sin": r = math.sin(_angulo(val))
        elif func_nome == "cos": r = math.cos(_angulo(val))

```

```

elif func_nome == "tan": r = math.tan(_angulo(val))
elif func_nome == "asin": r = _de_rad(math.asin(val))
elif func_nome == "acos": r = _de_rad(math.acos(val))
elif func_nome == "atan": r = _de_rad(math.atan(val))
elif func_nome == "log": r = math.log10(val)
elif func_nome == "ln": r = math.log(val)
elif func_nome == "sqrt": r = math.sqrt(val)
elif func_nome == "x2": r = val ** 2
elif func_nome == "x3": r = val ** 3
elif func_nome == "inv": r = 1 / val
elif func_nome == "exp": r = math.exp(val)
elif func_nome == "abs": r = abs(val)
elif func_nome == "fat": r = math.factorial(int(val))
elif func_nome == "10x": r = 10 ** val
else: return
if isinstance(r, float) and r.is_integer():
r = int(r)
s = str(round(float(r), 10))
if '.' in s:
s = s.rstrip('0').rstrip('.')
display_expr.set(f"{func_nome}({expressao}) =")
display_res.set(s)
ajustar_fonte(s)
expressao = s
except Exception:
display_res.set("Erro")
ajustar_fonte("Erro")
expressao = ""

```

```

def inserir_constante(c):
global expressao
if len(expressao) >= MAX_CHARS:
return
expressao += c
display_expr.set(expressao)
display_res.set(expressao)
ajustar_fonte(expressao)

```

```

def toggle_graus():
global modo_graus
modo_graus = not modo_graus
btn_deg.config(text="DEG" if modo_graus else "RAD",
bg=cor6 if modo_graus else "#c0392b")

```

```

def toggle_2nd():
global segundo_mod0
segundo_mod0 = not segundo_mod0
if segundo_mod0:
btn_sin.config(text="asin")
btn_cos.config(text="acos")
btn_tan.config(text="atan")
btn_2nd.config(bg=cor5)
else:
btn_sin.config(text="sin")
btn_cos.config(text="cos")

```

```

btn_tan.config(text="tan")
btn_2nd.config(bg=cor6)

def trig(): return "asin" if segundo_mod0 else "sin"
def trig_cos(): return "acos" if segundo_mod0 else "cos"
def trig_tan(): return "atan" if segundo_mod0 else "tan"

def tecla(event):
    k = event.keysym
    char = event.char
    if char in "0123456789": adicionar(char)
    elif char == ".": adicionar(".")
    elif char == "+": adicionar("+")
    elif char == "-": adicionar("-")
    elif char == "*": adicionar("*")
    elif char == "/": adicionar("/")
    elif char == "%": adicionar("%")
    elif char == "(": adicionar("(")
    elif char == ")": adicionar(")")
    elif k == "KP_0": adicionar("0")
    elif k == "KP_1": adicionar("1")
    elif k == "KP_2": adicionar("2")
    elif k == "KP_3": adicionar("3")
    elif k == "KP_4": adicionar("4")
    elif k == "KP_5": adicionar("5")
    elif k == "KP_6": adicionar("6")
    elif k == "KP_7": adicionar("7")
    elif k == "KP_8": adicionar("8")
    elif k == "KP_9": adicionar("9")
    elif k == "KP_Decimal": adicionar(".")
    elif k == "KP_Add": adicionar("+")
    elif k == "KP_Subtract": adicionar("-")
    elif k == "KP_Multiply": adicionar("*")
    elif k == "KP_Divide": adicionar("/")
    elif k in ("Return", "KP_Enter"): calcular()
    elif k == "BackSpace": apagar()
    elif k == "Escape": limpar()

janela.bind("<Key>", tecla)

def btn(parent, texto, linha, col, cmd,
        cspan=1, rspan=1, bg=cor4, fg=cor2):
    b = Button(parent, text=texto, bg=bg, fg=fg,
        font=("Ivy", 11, "bold"),
        relief=FLAT, overrelief=RIDGE,
        command=cmd, cursor="hand2",
        bd=1, highlightthickness=0)
    b.grid(row=linha, column=col, columnspan=cspan, rowspan=rspan,
        padx=1, pady=1, sticky="nsew")
    return b

frame_corpo = Frame(janela, bg=cor1, width=320, height=420)
frame_corpo.grid(row=1, column=0, sticky="nsew")
frame_corpo.grid_propagate(False)

```

```

for i in range(7):
frame_corpo.rowconfigure(i, weight=1)
for j in range(5):
frame_corpo.columnconfigure(j, weight=1)

btn_2nd = btn(frame_corpo, "2nd", 0, 0, toggle_2nd, bg=cor6)
btn_deg = btn(frame_corpo, "DEG", 0, 1, toggle_graus, bg=cor6)
btn(frame_corpo, "pi", 0, 2, lambda: inserir_constante(str(math.pi)), bg=cor6)
btn(frame_corpo, "C", 0, 3, limpar, bg=cor4)
btn(frame_corpo, "<-", 0, 4, apagar, bg="#c0392b")

btn(frame_corpo, "x2", 1, 0, lambda: aplicar_func("x2"), bg=cor6)
btn(frame_corpo, "1/x", 1, 1, lambda: aplicar_func("inv"), bg=cor6)
btn(frame_corpo, "|x|", 1, 2, lambda: aplicar_func("abs"), bg=cor6)
btn(frame_corpo, "exp", 1, 3, lambda: aplicar_func("exp"), bg=cor6)
btn(frame_corpo, "mod", 1, 4, lambda: adicionar("%"), bg=cor6)

btn_sin = btn(frame_corpo, "sin", 2, 0, lambda: aplicar_func(trig()), bg=cor6)
btn_cos = btn(frame_corpo, "cos", 2, 1, lambda: aplicar_func(trig_cos()), bg=cor6)
btn_tan = btn(frame_corpo, "tan", 2, 2, lambda: aplicar_func(trig_tan()), bg=cor6)
btn(frame_corpo, "n!", 2, 3, lambda: aplicar_func("fat"), bg=cor6)
btn(frame_corpo, "/", 2, 4, lambda: adicionar("/"), bg=cor5)

btn(frame_corpo, "x^y", 3, 0, lambda: adicionar("**"), bg=cor6)
btn(frame_corpo, "7", 3, 1, lambda: adicionar("7"), bg=cor4)
btn(frame_corpo, "8", 3, 2, lambda: adicionar("8"), bg=cor4)
btn(frame_corpo, "9", 3, 3, lambda: adicionar("9"), bg=cor4)
btn(frame_corpo, "x", 3, 4, lambda: adicionar("*"), bg=cor5)

btn(frame_corpo, "10x", 4, 0, lambda: aplicar_func("10x"), bg=cor6)
btn(frame_corpo, "4", 4, 1, lambda: adicionar("4"), bg=cor4)
btn(frame_corpo, "5", 4, 2, lambda: adicionar("5"), bg=cor4)
btn(frame_corpo, "6", 4, 3, lambda: adicionar("6"), bg=cor4)
btn(frame_corpo, "-", 4, 4, lambda: adicionar("-"), bg=cor5)

btn(frame_corpo, "log", 5, 0, lambda: aplicar_func("log"), bg=cor6)
btn(frame_corpo, "1", 5, 1, lambda: adicionar("1"), bg=cor4)
btn(frame_corpo, "2", 5, 2, lambda: adicionar("2"), bg=cor4)
btn(frame_corpo, "3", 5, 3, lambda: adicionar("3"), bg=cor4)
btn(frame_corpo, "+", 5, 4, lambda: adicionar("+"), bg=cor5)

btn(frame_corpo, "ln", 6, 0, lambda: aplicar_func("ln"), bg=cor6)
btn(frame_corpo, "+-", 6, 1, lambda: adicionar("*-1"), bg=cor4)
btn(frame_corpo, "0", 6, 2, lambda: adicionar("0"), bg=cor4)
btn(frame_corpo, ".", 6, 3, lambda: adicionar("."), bg=cor4)
btn(frame_corpo, "=", 6, 4, calcular, bg=cor5)

janela.mainloop()

```

CONCLUSÃO

Alcance dos Objetivos

O desenvolvimento da Calculadora Científica em Python com Tkinter permitiu ao grupo consolidar de forma prática os conhecimentos adquiridos ao longo da disciplina. Todos os objetivos propostos foram alcançados: a interface gráfica foi construída de maneira organizada e funcional, as operações matemáticas operam corretamente e o programa responde adequadamente a todas as entradas, seja por clique do mouse ou pelo teclado físico.

Desafios Encontrados

O maior desafio técnico foi a gestão de estado via variáveis globais, característica que exige atenção especial em Python. A implementação do modo 2nd, que altera dinamicamente os rótulos e comportamentos de três botões simultaneamente, também exigiu raciocínio cuidadoso sobre o fluxo de dados e eventos. O ajuste dinâmico de fonte no display demandou múltiplos testes para garantir legibilidade em diferentes comprimentos de resultado.

Aprendizados Técnicos

A equipe aprofundou seus conhecimentos em: criação de GUIs com Tkinter, programação orientada a eventos, uso da função `eval()` para avaliação de expressões matemáticas, tratamento de exceções com `try/except`, captura de eventos de teclado via `bind()`, organização de layout com o gerenciador `grid` e paleta de cores para interfaces. Esses conhecimentos são diretamente aplicáveis em projetos futuros de software.

Reflexão sobre o Processo

O trabalho em grupo foi essencial para a qualidade do resultado final. A divisão de responsabilidades entre os seis integrantes permitiu que cada módulo fosse desenvolvido e testado de forma independente antes da integração. Revisões coletivas contribuíram para a identificação de bugs e melhoria da legibilidade e documentação do código.

Considerações Finais

A Calculadora Científica desenvolvida é uma aplicação robusta, funcional e visualmente agradável. O projeto demonstra que é possível criar softwares úteis e bem estruturados com recursos nativos do Python. A experiência reafirmou a importância do planejamento prévio, da escrita de código limpo, dos comentários explicativos e do teste contínuo durante o desenvolvimento. O grupo considera que o projeto cumpriu plenamente seu papel educacional e prático, representando uma síntese dos aprendizados da disciplina de Desenvolvimento de Software no ano de 2026.

Contribuição Individual

Cada membro do grupo contribuiu ativamente nas diferentes etapas: definição do escopo, desenvolvimento do código, testes de funcionalidades, revisão de bugs e elaboração desta documentação. A experiência de trabalho colaborativo foi, em si mesma, um

aprendizado valioso sobre comunicação, divisão de tarefas e integração de soluções.

FONTES USADAS PARA A ELABORAÇÃO DA DOCUMENTAÇÃO E DO PROGRAMA

1. Documentação Oficial do Python

Python Software Foundation. *Python 3 Documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 2026.

2. Documentação do Tkinter

Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Disponível em: <https://docs.python.org/3/library/tkinter.html>. Acesso em: 2026.

3. GeeksforGeeks — Python Tkinter

GeeksforGeeks. *Python Tkinter Tutorial*. Disponível em: <https://www.geeksforgeeks.org/python-tkinter-tutorial/>. Acesso em: 2026.

4. Módulo math do Python

Python Software Foundation. *math — Mathematical functions*. Disponível em: <https://docs.python.org/3/library/math.html>. Acesso em: 2026.

5. Real Python — GUI com Tkinter

Real Python. *Python GUI Programming With Tkinter*. Disponível em: <https://realpython.com/python-gui-tkinter/>. Acesso em: 2026.

6. Stack Overflow — Comunidade Python

Stack Overflow Community. *Questões sobre Tkinter e Python*. Disponível em: <https://stackoverflow.com/>. Acesso em: 2026.

7. TkDocs — Tutorial Tkinter Moderno

Roseman, M. *TkDocs: Modern Tkinter for Busy Python Developers*. Disponível em: <https://tkdocs.com/>. Acesso em: 2026.

8. W3Schools — Python

W3Schools. *Python Tutorial*. Disponível em: <https://www.w3schools.com/python/>. Acesso em: 2026.